

Web design for the Sony PSP (*PlayStation Portable*)

Published: 31st October 2005

Last updated: 10th May 2006

This article is an introduction to designing and building web pages for the Sony PSP (*PlayStation Portable*). We'll be looking at the browser's main features, outlining its support for various web technologies, and demonstrating a range of useable techniques and hacks.

- The browser view
- Navigating pages
- Fonts
- HTML and embedded content
- CSS support
- JavaScript support
- Flash support
- Generating PSP-specific content
- btw: Copying movies to the PSP
- Further reading



To inform and accompany the article I made a range of test pages and other demos:

- View the tests online
- Download a zipfile of the tests [66K]

The browser view

The PSP (*PlayStation Portable*)'s screen is 480× (by) 272, which gives it an aspect ratio of 16:9 - comparatively wider than the 4:3 (16:12) ratio we're typically used to designing for. In absolute terms of course it's quite small, but nothing like as small as a typical PDA or Smartphone, so really it can't be considered a small-screen device - it's a mid-screen device, more like a TV than a typical handheld. The color-depth is a delicious 24-bit - no surprise in a device that's primarily for games.

The browser has a two-part chrome (an address bar at the top, and controls at the bottom), but it's not a permanent part of the browser view - it's retractable, using the Triangle button, and translucently *overlaid on top* of the page view when it is present. So ultimately, the browser can be considered as though it's **always in fullscreen mode** - with no chrome to account for when designing.

Navigating pages

There are two controls for navigating around a page: the D-Pad (*Directional Paddle*) behaves like a kind of two-dimensional Tab key - it moves between focusable elements in four directions (very much like Spatial Navigation in Opera); the analog-stick behaves like a basic mouse or trackpad, allowing for cursor movement to any part of the screen. Links and controls can be clicked by pressing the "action" button, which is either X or Circle, depending on the regional model (I have a Japanese one, which uses Circle).

If you hold down the Square button the controls change their function - they'll now scroll the whole page, instead of moving the cursor. In this case the analog-stick acts like mouse-wheel scrolling (but in four directions, if necessary), while the D-Pad (*Directional Paddle*) arrows act like Page-up and Page-down buttons (ditto).

Paging with the D-Pad (*Directional Paddle*) moves the page by **exactly one screen** each time, so content which is arranged in blocks of *exactly the right size* will come out as a kind of "paged" content ["paged" content demo]. Single pages of the right size can be used like modal dialogs, or for step-by-step processes such as a checkout or application form [dialog page demo]

When holding down the Square button, a colored strip appears at the very top of the page, to indicate the change in mode; this strip is 9px high.

Fonts

The browser has **five** font-sizes and **two** fonts. Its fonts are **serif** and **sans-serif**, and the latter is *always* used unless a **serif** is specified [fonts demo]

Font-sizes are in the following range:

- <= 0.85em / <= 12px (small)
- 0.9em - 1.1em / 13px - 16px (medium [default])
- 1.15em - 1.3em / 17px - 19px (large)
- 1.35em - 1.85em / 20px - 27px (x-large)
- >= 1.9em / >= 28px (xx-large)

On the PSP (*PlayStation Portable*) as with all devices, a serif font will look noticeably smaller than a sans-serif font of the same type-size. But since there are only five sizes to choose from, it won't be possible to make small adjustments to compensate, so it's probably best to avoid the smallest size for serif fonts.

The browser has a "view > text-size" option with three settings: "small", "medium" and "large"; the default setting is "medium". The view sizing cannot scale outside the overall range, so for example, with a view setting of "small" the two smallest font-sizes appear the same.

The default size is nice and comfortable to read on the PSP (*PlayStation Portable*), yet most websites specify screen fonts that are **too small** by default - they're aimed at desktop and laptop computers (also screen-media devices), but they take the PSP (*PlayStation Portable*) right down to its smallest size, which is readable, but not comfortable for long.

What I recommend you do is **increase the overall font-size**, just for the PSP (*PlayStation Portable*), which you can implement like this:

```
body { font-size:100%; }
html:not(:nth-child(n)) body { font-size:125%; }
```

Then any subsequent font-size rules, such as 0.8em for paragraphs, will even-out to a size that's comfortable for both. For more about this particular hack, please see the later section on CSS hacks.

HTML and embedded-content

The browser supports most HTML elements (including XHTML syntax), but quite a lot of meta-data will be lost on it - abbreviations and definitions, for example, are no use without a means of displaying their `title` attribute, but the PSP (*PlayStation Portable*) **doesn't display titles**. It will display `alt` text for images, but only if the image path is broken (quite rightly).

Framesets are supported, and so are `<iframe>` elements, however in the latter case the browser never seem to render scrollbars - even with the `scrolling` attribute set and using `overflow:scroll`. But you can still scroll freely within an `iframe` document, by moving the cursor inside it and using Square plus the analog stick; you can also navigate between elements inside it, using the D-Pad (*Directional Paddle*) as normal. (`<textarea>` elements do have a scrollbar when necessary, but it's a tiny little slip of a thing that's extremely difficult to use.)

The `<object>` element is supported, and can be used to embed any other content of a compatible mime-type. I've confirmed that the following are supported: `text/html` (web pages), `image/gif` (GIF (*Graphics Interchange Format*) images), `image/jpg` (JPEG (*Joint Photographic Experts Group*) images), `image/png` (PNG (*Portable Network Graphics*) images, including alpha-channel transparency). Flash support was added in the 2.7 firmware update [`<object>` tests]

Notable mime-types that are currently unsupported include `text/plain` (text documents), `text/xml` (XML (*eXtensible Markup Language*) documents) and `application/rss+xml` (RSS (*Really Simple Syndication*) feeds, although the PSP (*PlayStation Portable*) does have an "RSS Channel" in the "Network" menu, this is only for enclosure-based feeds like podcasts, and doesn't appear to support plain information feeds; but if podcasts are your thing, the 2.7 update adds the ability to save enclosures to the memory stick).

The browser doesn't support `application/xhtml+xml`, but neither does it choke on pages served like that.

CSS (*Cascading Style Sheets*) support

The browser's CSS (*Cascading Style Sheets*) support is **awesome!** Most of the CSS layouts I've looked at have been basically fine - a few glitches here and there, but nothing major to cry about. Most of the problems I've seen come down to **whether the design can cope with the restricted space**. With that caveat, I think it's safe to work on the basis that a valid, well-built layout will be basically fine on the PSP (*PlayStation Portable*).

A layout based on **percentage widths** stands the best chance of working in all devices, but of course for PSP (*PlayStation Portable*) -specific sites you have the freedom to work with a fixed, predictable page size.

I haven't done an extensive amount of testing to determine the details of its support, but from what I have done I can give you a few tasty facts:

- The browser is a **screen media** device
- It supports an impressive chunk of CSS1 and CSS2, including attribute selectors, **@media** and **@import**
- It supports **:hover**, **:focus**, **:active** and **:visited** on links (but none on any other elements); since hover and focus states are both generated, their rules will be applied according to precedence
- It implements the standard box model (width = content-box-width) and doesn't implement doctype switching

CSS hacks

The browser has a few interesting parsing bugs that give rise to useable CSS hacks, and the most notable among these is its treatment of **negation**. For example, the **negation pseudo-class** <http://www.w3.org/TR/2001/CR-css3-selectors-20011113/#negation> (defined in CSS3 (*Cascading Style Sheets Level 3*)) is *not* supported, however it is sometimes *ignored* as though it weren't there; specifically: **a descendent-selector from a negated selector is applied irrespective of the negation** [CSS negation tests]

The browser's behavior with negation as part of an **@import** or **@media** statement is the same, so taking these behaviors together, we find that the following CSS hacks can be used to identify the PSP (*PlayStation Portable*) :

1. **@import with negated media type** http://www.dithered.com/css_filters/css_only/import_with_not_media.html
2. **@media with negated media type** http://www.dithered.com/css_filters/css_only/media_not_all.html
3. **The "not-my-child" hack** http://www.dithered.com/css_filters/css_only/not_my_child.html (if used as part of a descendent selector)

But the first two hacks *also* apply to Pocket IE for Windows Mobile, which may be an issue if your styles are targetted at small-to-mid-screen devices generally. Of the three hacks listed above, **only the third will apply uniquely to the PSP (*PlayStation Portable*)**, so that would be my general recommendation for spot tweaks and fixes:

```
#selector { ... general rules ... }
html:not(:nth-child(n)) #selector { ... PSP rules ... }
```

(Actually this hack also applies to MSN for OSX, but that really is a dead browser - officially withdrawn by Microsoft - so nobody, but nobody will be using it for real!)

This hack is fine for small adjustments, but if you want to apply a lot of rules specifically to the PSP (*PlayStation Portable*), or design a layout customized just for it, I recommend you feed it a special stylesheet using one of the techniques for generating PSP-specific content.

JavaScript support

It took me a while to find some, but it's there if you dig deep enough!

Unfortunately there's quite a gulf between what the browser reports as supporting, and what it actually does support; iterating through **window** and other objects can provide a technical overview, but only detailed investigation can reveal how much of that is really useable. The differences might be down to the fact that **Sony licensed the browser** <http://www.psp-vault.com/modules.php?op=modload&name=News&file=article&sid=244> from another company, so there may well be capabilities in the original product that aren't implemented in this version.

The most significant gaps are in support for DOM methods like **createElement** and **appendChild**, which are reported as being supported, but created elements return **null**, and can't be added to the page. In the PSP (*PlayStation Portable*) browser then, **it isn't possible to add to the DOM (*Document Object Model*) after load time** - neither DOM (*Document Object Model*) element creation nor **innerHTML** will work - all you can do to add HTML (*HyperText Markup Language*) is **document.write** during page construction [dynamic-content tests]

I don't know where the root of this limitation actually lies - whether the methods aren't implemented, or whether the browser simply doesn't redraw.

Initializing scripts

The PSP (*PlayStation Portable*) browser supports *four* methods of initializing a script at load time:

1. the `window.onload` event
2. a hard-coded `<body onload>` attribute
3. a hard-coded `<img onload>` attribute (for example, placed at the end of the page to act like a document load event)
4. a timeout-based solution which checks for the existence of DOM (*Document Object Model*) objects, such as my **domFunction helper script** `<http://www.brothercake.com/site/resources/scripts/domready/>`

Despite what I said in the first version of this article, `window.onload` actually is supported, and always has been. It turns out that its implementation has an extremely bizarre behavioral quirk...

When a page contains both a `window.onload` *and* a `<body onload>`, in most browsers only one of them will fire - the one that was defined last. (The exception to this is Opera, which treats them as two, separate events.)

The PSP (*PlayStation Portable*) browser follows the norm, and only fires one event, however `window.onload` *won't always take precedence, even if it was defined last*. Specifically: `<body onload>` **will take precedence over** `window.onload`, irrespective of the source order, **if the head section contains a non-empty style or link element**.

I told you it was bizarre! It's literally a case of whether the elements are present and sufficiently defined that they *could* import some CSS (*Cascading Style Sheets*), even if they don't actually do so! So a completely empty `<style>` tag pair, or a `<link>` element with only a single attribute (not enough to define an import), won't trigger this quirk, and `window.onload` will take precedence as expected [window.onload test 1 - works as expected]:

```
<link rel="stylesheet" />

<link href="non-existent.css" />

<link href="existent.css" />

<style></style>

<style type="text/css" media="screen, projection"></style>
```

But if there's anything inside the `<style>` element (even whitespace) or enough attributes in the `<link>` to define a stylesheet (even if the file doesn't exist, or is empty), then that's enough to trigger this quirk, and `<body onload>` will take precedence [window.onload test 2 - `<body onload>` takes precedence]:

```
<link rel="stylesheet" href="non-existent.css" />

<link rel="stylesheet" href="existent.css" />

<link rel="stylesheet" type="text/css" href="existent.css" media="screen, projection" />

<style>
</style>

<style>                </style>

<style>/**/</style>
```

To plot a route through this mess, I recommend first and foremost that you avoid using the `<body onload>` attribute yourself, which is sound advice quite apart from this issue, for the sake of good separation between content and logic.

Over and above that, you can determine whether it's safe to use `window.onload` by the following reasoning:

If... you're developing on a page you control completely, or where you know no other scripting will be used you can safely use `window.onload`; if there won't ever be any `<body onload>` to deal with, this issue will never come up

Otherwise...

don't use `window.onload` at all; instead, use the final technique I mentioned for initializing scripts - a **timeout-based solution** <<http://www.brothercake.com/site/resources/scripts/domready/>>

`window.onload` has its own problems quite apart from this issue, in that a page can only contain one such function (unless you use a **closure-based solution** <<http://simon.incutio.com/archive/2004/05/26/addLoadEvent>>). You may feel that it's easier just to avoid it altogether, and in that case the timeout-based solution would remain the optimum choice.

Event handling

Once you're hooked in, you can add event handlers using DOM0 (*Document Object Model "Level 0"*) expandos, and the following events are **confirmed to work for links and form controls**:

Using the D-Pad (*Directional Paddle*) or analog-stick

`onmouseover`, `onmouseout`, `onmousemove`, `onfocus`, `onblur`

Using the action button

`onmousedown`, `onmouseup`, `onclick`

It may well be possible to handle some of these events on other, non-interactive elements, but I haven't checked that far. I figure there's little point in scripting for elements other than links and form widgets, since those are the only ones that are accessible to the D-Pad (*Directional Paddle*) .

Events from the D-Pad (*Directional Paddle*) and analog-stick are generated almost identically, except that the analog-stick generates more `MOUSEMOVE` events - they happen continually, where the D-Pad (*Directional Paddle*) only generates one or two each time the focus moves to an element. Key events are not generated.

You can use hard-coded attribute handlers for any of these events, but as we know, this is not recommended. Neither `addEventListener` nor `attachEvent` are supported, so expandos are the way to go!

Practical capabilities

Next we'll need to know what we can *actually do* in scripts, and the oldskool basics are all there - `window` methods like `open()` and `setTimeout`, document properties and collections such as `cookie` and `forms[]`, arrays, strings, numbers ... and so on. Also supported are DOM (*Document Object Model*) getters such as `getElementById`, `getElementsByTagName` and `getAttribute` (including the usual attribute-property mappings like `element.className`).

But for visual output the choices are very limited - it's not possible to add new content to the DOM (*Document Object Model*) , as we've already noted, and in fact almost no dynamic modification is supported at all - we can't change the display of an element, nor even the `SRC` of an image!

What we *can* do is change the **visibility or position** of an object, allowing for toggles and basic animation, and also the **background-position** of an element with a CSS (*Cascading Style Sheets*) background image, which allows for image effects based on **pixy's method** <<http://wellstyled.com/css-nopreload-rollovers.html>> (and from the evidence of the games I've made, this is the fastest technique).

With the textual output afforded by writing to text fields and other form elements (also supported fine), and the nicely-styled interface possibilities afforded by `<input type="image">` and judicious use of 24-bit PNGs (*Portable Network Graphics*) , it's possible to write some pretty decent games (and other apps ... but let's face it, games are the scripting of choice in an environment like this!). For some examples check out my selection, and the large library available at **PSP Web Browser** <<http://www.pspwebbrowser.com/>>.

Overall, this browser really reminds me of Opera 5 - you can do dynamic stuff, but only with content that's already generated.

RPC hooks

First and foremost, it doesn't support `XMLHttpRequest`, which is disappointing but hardly surprising. Loading data through an `<iframe>` is also problematic, because although the element itself is supported, you can't change its `SRC` through script; or at least, it appears as though you can't because the page doesn't change, but if you set and then query the `SRC` property, it does indeed come back with the new value! Is it updating but not re-drawing? Again, I couldn't tell you. [JS loader test 1 - failed]

But we know that iframes work for normal navigation - if we click a link that's targeted to an iframe, the contents will update just fine, and the same thing is true for **submitting a form to an iframe**. If we can do that, all we need then is a means of making it

happen automatically, and happily this is possible via the programmatic `submit()` method - we can programmatically submit a form, and that will give us a dynamic method for updating the contents of the iframe. Phew!

Finally then, we can come full circle - submit to an iframe, and then have a function in the resulting page which sends data back up to the parent. [JS loader test 2 - success!]

Flash support

The Flash player is new in the 2.7 firmware upgrade, and a keenly-anticipated, and most welcome addition it is! This could really be the thing that takes PSP (*PlayStation Portable*) application development to the next level.

The player is disabled by default; to turn it on you need to go to “Settings > System Settings” (within the PSP (*PlayStation Portable*)'s main menu) and select “Enable Flash Player”. You have to agree to an EULA (*End-User License Agreement*) and connect to the internet to confirm, and then the browser can view Flash! [Flash demo]

Once it's enabled, the PSP (*PlayStation Portable*) supports Flash content embedded using the **Flash-satay**

<<http://www.alistapart.com/articles/flashesatay/>> method:

```
<object type="application/x-shockwave-flash" data="content.swf" width="114" height="81">
  <param name="movie" value="content.swf" />
  <p>Redundent content</p>
</object>
```

It also supports the traditional `<embed>` convention, but this is obviously not recommended since it uses invalid markup:

```
<embed type="application/x-shockwave-flash"
  pluginspage="http://www.macromedia.com/go/getflashplayer"
  src="content.swf" width="114" height="81"/>
```

Detecting the Flash player can be done in JavaScript. The browser reports the `navigator.plugins` collection, containing the `["Shockwave Flash"]` object; this contains the `description` property, which returns the value `"Shockwave Flash 6.0 r72"`. [Flash detection demo]

This tells us two things:

1. The Flash player is Version 6
2. It can be detected in the same way as for other browsers, using a generic script like **PPK's**
<<http://www.quirksmode.org/js/flash.html>>

But although the player is Version 6, it's missing quite a chunk of features compared with its desktop sibling; Bill Perry of Adobe has compiled an **overview of what's available** <<http://www.flashdevices.net/2006/04/sony-psp-flash-player-6-supported.html>>.

Generating PSP (*PlayStation Portable*) -specific content

There are server-side and client-side means of identifying and feeding content specifically to the PSP (*PlayStation Portable*), but since the client-side methods are based around `document.write`, I don't recommend their use unless you have no alternative. I would recommend using the server-side methods wherever possible.

Server-side

The browser's user-agent string is `"Mozilla/4.0 (PSP (PlayStation Portable); 2.00)"`, so any unique part of that value can be used as a condition for writing content to the page:

```
if(strstr($_SERVER['HTTP_USER_AGENT'], 'PlayStation Portable'))
{
    echo('<link rel="stylesheet" href="psp.css" type="text/css" />');
}
```

The numbers in the UA (*User-Agent*) string suggest that it came from a device running the 2.0 firmware, but in fact this value is constant - it always says "2.00" irrespective of the version.

The PSP (*PlayStation Portable*) browser also sends *two unique headers* with its requests, which are "http_x_psp_browser" and "http_x_psp_productcode", however these were added *after* the 2.0 firmware update which introduced the browser itself, and are **not available** in that version. I know they were present in 2.5, and there was only one release in between those two (a 2.01 security update). So I assume these values were added in 2.5, but I don't know that for sure, and all this makes them far less useful for content inclusion than I previously thought (and said!).

But they may still have a use if you specifically need to detect 2.5 or later versions. The actual value returned by "http_x_psp_browser" will be something like "2.70 (LX)", so the number tells us the firmware and can be used to identify specific versions (such as 2.7 or later, which have the browser Flash plugin, although of course this doesn't tell us whether that feature is actually enabled, for which we need JavaScript detection). The value of "http_x_psp_productcode" is "J1" in my case, so I assume that it's the region code, since mine is a Japanese model.

Client-side

We can use the user-agent string for a JavaScript condition, just as we did for PHP (*HyperText Pre-Processor*) :

```
if(navigator.userAgent.indexOf('PlayStation Portable') != -1)
{
    document.write('<link rel="stylesheet" href="psp.css" type="text/css" />');
}
```

btw: Copying movies to the PSP

I'm wandering slightly off-topic here, but I wanted to add a quick note about copying movies to the PSP (*PlayStation Portable*) - you can, but it's very picky about *filename conventions*, and also about *where* the video is stored on the memory stick - get either aspect wrong and it simply won't show up!

You can take the pain out of this using a shareware application called **PSP Movie Creator** <<http://www.pqdvd.com/psp/>>, which can convert various file-formats, rip DVDs, and then transfer them directly to your PSP (*PlayStation Portable*) . I ripped "Star Wars Episode IV (*Four*) " at a high quality setting, and it came out just under 800MB (800 mega-bytes) - so a 1GB (1 giga-byte) card should be enough to accomodate most single movies (or 4 to 5 episodes of Star Trek)

And I've recently discovered an application called **iPSP** <<http://ipsp.kaisakura.com/>>, which is a complete multi-media management and backup tool for your PSP (*PlayStation Portable*) , available for Mac OS X (*Macintosh Operating System Version 10*) or Windows. I've only played with a little bit so far, but it's looking very cool indeed

Further reading

- **Designing Websites and CSS for Sony's PSP** <http://www.neuromantics.net/bunker/arc/design/designing_websites_and_css_for_sonys_psp.shtml> [neuromantics.net]
- **Yep, that's a NetFront browser you're using** <<http://www.psp-vault.com/modules.php?op=modload&name=News&file=article&sid=244>> [psp-vault]
- **Designing for TV** <<http://developer.msntv.com/Designing/>> [msntv.com]
- **Pocket-Sized Design: Taking Your Website to the Small Screen** <<http://www.alistapart.com/articles/pocket>> [a list apart]

[brothercake] <<http://www.brothercake.com/>> *a round peg in a square hole, that still fits*